

HAFTA

03

Tarayıcı ve HTML

Sunucudan gelen şey bir metin dosyasıydı. Bu hafta o metnin nasıl önce bir ağaca, sonra ekranda piksellere dönüştüğüne bakıyoruz.

HAFTANIN SORUSU

*Tarayıcı bir metin dosyasını nasıl
ekrana çeviriyor?*

Bu haftanın hedefleri

Her başlığın derinliği aynı değil

✓ MUTLAKA BİL

HTML belge iskeletini, anlamsal öğeleri ve kaynak kod-DOM ayrımını açıkla.

>_ DERSTE UYGULA

Yalnızca div kullanan bir sayfayı anlamsal HTML ile yeniden yaz.

🔍 MERAK EDİYORSAN İNCELE

Ayrıştırma sırası, tarayıcının hatalı HTML'i onarması ve CSSOM ilişkisi.

Kampüs panosu: elimizdeki tek şey bir metin dosyası

```
<h1>Kampüs Panosu</h1>
<article>
  <h2>Yeni laboratuvar açıldı</h2>
  <p>Tanıtım toplantısı 14.00'te.</p>
</article>
```

Sunucudan gelen tam olarak budur: biçimlendirilmemiş düz metin. Ekranda ise başlık büyük, duyuru onun altında, satırlar ayrı ayrı görünür. Aradaki dönüşümü bugün adım adım izleyeceğiz.

SORU

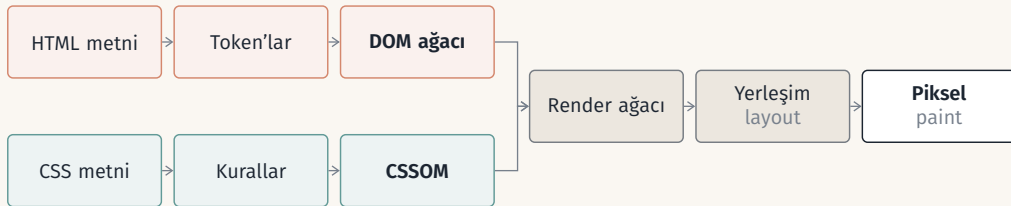
Başlığın ekranda büyük görünmesi, onun h1 olduğu anlamına gelir mi?

1

Tarayıcının oluşturma süreci

Tarayıcı HTML ve CSS'i işleyerek sayfayı ekrana çizer.

Metinden piksele



Bu hafta soldaki üst şeridi işliyoruz: **HTML metninden DOM ağacına**. Alt şerit önümüzdeki haftanın konusu.

- Tarayıcı HTML'i **yukarıdan aşağıya** okur ve okudukça DOM'u kurar.
- `defer` ya da `async` taşımayan bir `<script>` ile karşılaşırsa ayrıştırma **durur** — betik çalışmadan devam etmez.
- Çünkü betik DOM'u değiştirebilir; tarayıcı ne olacağını önceden bilemez.

Bu yüzden betikler geleneksel olarak sayfanın **sonuna** konur ya da `defer` ile işaretlenir: “beni sonra çalıştır, sen okumaya devam et”.

→ **WP-II** React uygulamaları da sonunda bu hattan geçer; yalnızca DOM'u JavaScript kurar.

2

HTML'in yapısı

Her HTML belgesi aynı iskeleti paylaşır.

Belgenin iskeleti

```
<!doctype html>
<html lang="tr">
  <head>
    <meta charset="utf-8">
    <title>Kampüs Panosu</title>
  </head>
  <body>
    <h1>Kampüs Panosu</h1>
  </body>
</html>
```

- head ekranda görünmeyenler: başlık, kodlama, stil bağlantıları
- body kullanıcının gördüğü her şey
- charset karakter kodlaması — eksikse Türkçe harfler bozulur
- lang belgenin dili — ekran okuyucu ve çeviri bunu kullanır

Bir öğenin (element) anatomisi

`<a href="/hakkimizda">Hakkımızda</a>`

etiket adı nitelik içerik kapanış etiketi

Nitelikler (attribute) öğeye **ek bilgi** verir. Bazıları her öğede kullanılabilir (id, class, lang), bazıları yalnızca belirli öğelerde anlamlıdır (href, src, alt).

Sık kullanılan öğeler

METİN VE BAĞLANTI

- h1–h6 başlık hiyerarşisi
- p paragraf
- a bağlantı
- strong, em vurgu
- ul, ol, li listeler

İÇERİK VE YAPI

- img görsel (alt zorunlu sayın)
- table, tr, th, td
- div anlamsız kutu
- span anlamsız satır içi kutu
- form 5. hafta

i BAŞLIK HİYERARŞİSİ

h1–h6 bir **içindekiler ağacıdır**, font boyutu ayarı değildir. Atlamadan kullanın; görsel boyut CSS'in işidir.

3

Anlamlı HTML

Aynı görüntüyü veren iki belgeden biri makineler için anlamsızdır.

İki belge, aynı görüntü

ANLAMSIZ

```
<div class="ust">
  <div class="menu">...</div>
</div>
<div class="icerik">
  <div class="yazi">
    <div class="baslik">Haber</div>
  </div>
</div>
```

ANLAMLI

```
<header>
  <nav>...</nav>
</header>
<main>
  <article>
    <h1>Haber</h1>
  </article>
</main>
```

Ekranda ikisi de aynı görünebilir. Ama sağdaki belge **ne olduğunu söylüyor**; soldaki yalnızca **nasıl görüldüğünü** ima ediyor.

Bir sayfanın anlamlı iskeleti



Bu öğeler **anlam taşır**; tarayıcı varsayılan stiller uygulayabilir. Görünüm önümüzdeki haftanın işi.

Anlamsal HTML neden önemli?

- **Ekran okuyucular** “ana içeriğe atla” diyebilmek için `main` gibi bir işarete ihtiyaç duyar; yalın `div` bu rolü belirtmez.
- **Arama motorları** `article` ve `h1` ile sayfanın neyle ilgili olduğunu çıkarır.
- **Tarayıcı özellikleri** okuma modu, içindekiler çıkarma, otomatik çeviri — hepsi anlamdan beslenir.
- **Siz** altı ay sonra kendi kodunuzu okuduğunuzda `<div class="k2">` ne demekti diye düşünmezsiniz.

Birlikte çözelim: panoyu anlamlandıralım

ELİMİZDEKİ

```
<div class="ust">
  <div class="b">Kampüs Panosu</div>
</div>
<div class="icerik">
  <div class="duyuru">
    <div class="bas">Yeni lab</div>
    <div>Toplantı 14.00</div>
  </div>
</div>
```

HANGİ ÖĞE, NEDEN

ust → header sayfa başı

b → h1 sayfanın başlığı

icerik → main asıl içerik

duyuru → article tek başına anlamlı

bas → h2 duyurunun başlığı

son div → p paragraf

Öğeyi görünümüne göre değil, **sayfadaki rolüne göre** seçin — rolü olmayan bir kutu için div doğru tercihtir.

Sıra sizde

🕒 ALIŞTIRMA · 20 dk

Size verilen, tamamı `div` ile yazılmış sayfayı anlamsal öğelerle yeniden yazın.

1. Her `div`'in yerine hangi öğenin geleceğine karar verin
2. Başlık hiyerarşisini `h1`'den başlayarak düzeltin
3. Görsellere `alt` metni ekleyin
4. **F12 → Elements** ile sonucu kontrol edin

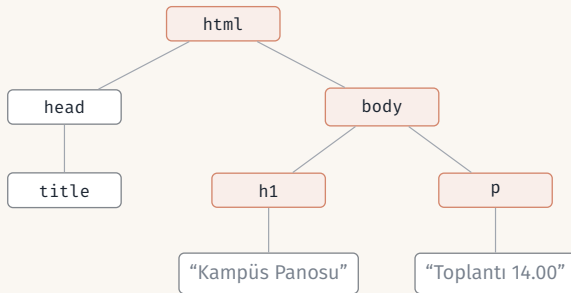
Bitirdikten sonra şu soruyu yanıtlayın: **Sayfanın görüntüsü değişti mi? Neden?**

4

DOM

Metin bir belgedir; DOM programlanabilir bir nesnedir.

HTML bir metindir, DOM bir ağaçtır



Her öge bir **düğüm**, her metin parçası da ayrı bir düğümdür — en alttaki iki kutu metin düğümüdür. JavaScript bu ağaçta gezinir, düğüm ekler, siler, değiştirir; tarayıcı ekranı buna göre günceller.

Kaynak kod ile DOM aynı şey değil

>_ CANLI DEMO

Aynı sayfayı iki yerde açalım: kaynak kodu (Ctrl+U) ve **F12 → Elements**. İkisi neden farklı görünüyor?

- Kaynak kod **sunucudan gelen metindir** — hiç değişmez.
- Elements sekmesi **canlı DOM'u** gösterir — JavaScript onu değiştirebilir.
- Eksik kapatılmış bir etiket kaynakta hatalı, DOM'da **düzeltilmiş** görünür: tarayıcı bozuk HTML'i onarmaya çalışır.

Üç soru

Her yanıtı bir gerekçeyle birlikte verin

1. Etiket ile öge arasındaki fark nedir?
2. charset ve lang aynı işi mi yapar?
3. Elements panelinde yaptığınız bir değişiklik, sayfayı yenileyince neden kaybolur?

Haftanın sorusunun cevabı

1. Sunucudan **düz metin** gelir (Content-Type: text/html).
2. Tarayıcı metni soldan sağa, yukarıdan aşağıya **ayrıştırır**.
3. Etiketlerden **DOM ağacını** kurar; bozuk yapıları onarmaya çalışır.
4. CSS'ten ayrı bir ağaç (CSSOM) kurar ve ikisini birleştirir.
5. Yerleşimi hesaplar ve pikselleri **boyar**.

Bugün 1–3'ü işledik. **4 ve 5** önümüzdeki haftanın konusu.

Nerede kaldık?

Üç hafta sonra: belge yapısı yerine oturdu

01 Ağ ve DNS	02 HTTP	03 HTML ve DOM	04 CSS
05 Formlar	06 Git ve yayın	07 JavaScript	08 DOM ve olaylar
09 Asenkron ve JSON	10 Fetch ve API	11 Durum ve önbellek	12 Backend
13 Veritabanı	14 Güvenlik		

Kiremit çerçeve: işlendi. Gri çerçeve: sonraki haftalar.

Ağaç kuruldu ama her şey üst üste, siyah beyaz duruyor. **Görünümü kim belirliyor?**

CSS'in nasıl karar verdiğini, kutu modelini ve aynı sayfanın telefonda neden farklı dizildiğini göreceğiz.

→ WEB PROGRAMLAMA II

React bileşenleri sonuçta bu DOM ağacını üretir. Bu haftayı bilmeden React'ın ne yaptığı anlaşılmaz.